DECISION GOVERNANCE INFRASTRUCTURE

UNITED STATES PATENT AND TRADEMARK OFFICE

PROVISIONAL PATENT APPLICATION

TITLE OF INVENTION:

CALIBRATION DRIFT DETECTION AND EXECUTION BLOCKING SYSTEM FOR AUTOMATED DECISION GOVERNANCE WITH TEMPORAL SNAPSHOT COMPARISON, WEIGHTED MULTI-SIGNAL DRIFT SCORING, AND FAIL-CLOSED CERTIFICATION POLICY

Docket Number:	OMNIX-PAT-2026-004
Applicant / Inventor:	Harold Alberto Nunes Rodelo
Nationality:	United Kingdom
Entity Status:	Micro Entity (37 C.F.R. § 1.29)
Filing Basis:	35 U.S.C. § 111(b) — Provisional Application
Date Prepared:	April 19, 2026
Date of Filing:	April 19, 2026

This document constitutes a Provisional Patent Application filed pursuant to 35 U.S.C. § 111(b). It establishes a priority date and grants twelve (12) months from the filing date to submit a corresponding non-provisional application. This application has not been examined and does not confer patent rights independently. CONFIDENTIAL — NOT FOR DISTRIBUTION.

PROVISIONAL PATENT APPLICATION

OMNIX-PAT-2026-004

Title: CALIBRATION DRIFT DETECTION AND EXECUTION BLOCKING SYSTEM FOR AUTOMATED DECISION GOVERNANCE WITH TEMPORAL SNAPSHOT COMPARISON, WEIGHTED MULTI-SIGNAL DRIFT SCORING, AND FAIL-CLOSED CERTIFICATION POLICY

Inventor: Harold Alberto Nunes Rodelo

Applicant: OMNIX QUANTUM LTD

Filing Basis: 35 U.S.C. § 111(b)

Entity Status: Micro Entity

Date Prepared: April 19, 2026

Related Applications: OMNIX-PAT-2026-001 (Governance Control Architecture)

FIELD OF THE INVENTION

The present invention relates to automated decision governance systems, and more particularly to a system and method for detecting calibration drift in machine learning models and automated decision pipelines, comparing current operating conditions against a stored calibration baseline, computing a weighted multi-signal drift score, and enforcing a fail-closed blocking policy when calibration assumptions are no longer valid.

BACKGROUND

I. THE KNIGHTIAN UNCERTAINTY PROBLEM IN AUTOMATED GOVERNANCE

Automated decision systems — including machine learning models, algorithmic trading engines, clinical decision support systems, and autonomous control systems — are calibrated under a specific set of conditions at a specific point in time. The calibration process establishes the parameters, thresholds, and behavioral assumptions that govern the system's subsequent operation. These calibrated assumptions constitute the evidential basis for any governance certification issued by the system: when the system certifies that a proposed action is admissible, it implicitly certifies that the calibration assumptions underlying that certification remain valid.

The fundamental problem is that calibration conditions change. Market regimes shift. Patient populations evolve. Environmental conditions vary. Regulatory requirements are updated. When the conditions under which a system was calibrated diverge significantly from the conditions under which it is currently operating, the governance certifications issued by that system become unreliable. A system may correctly certify a proposed action given its calibrated parameters while simultaneously being miscalibrated for the current environment — producing a false certification that appears structurally valid but is substantively invalid.

II. INADEQUACY OF EXISTING APPROACHES

Existing automated decision systems fail to address this problem in three distinct ways:

2.1 Static Calibration: Most systems are calibrated once and deployed indefinitely without any mechanism for detecting whether calibration assumptions remain valid. The system continues to issue certifications regardless of how much the operating environment has changed since calibration.

2.2 Periodic Recalibration Without Blocking: Some systems implement periodic recalibration schedules (e.g., monthly model retraining). However, these systems do not block execution during the interval between calibration events, even

when real-time conditions have diverged dramatically from the most recent calibration baseline.

2.3 Silent Failure Under Non-Finite Inputs: Existing systems routinely fail silently when governance signals contain NaN (Not a Number) or Inf (infinity) values. In Python and similar languages, the comparison `NaN > threshold` evaluates to `False`, causing a governance system to silently pass a proposed action that should be blocked due to a corrupted or missing signal. No known prior art system explicitly detects and blocks on non-finite signal inputs as a first-priority checking condition.

2.4 No Certification Provenance: Existing systems do not link governance certifications to the specific calibration snapshot under which they were issued, making it impossible to retrospectively invalidate certifications issued under a calibration baseline that was subsequently determined to be invalid.

The present invention addresses all four inadequacies through the Assumption Validity Monitor (AVM) architecture.

SUMMARY OF THE INVENTION

The present invention provides a system and method for monitoring the continued validity of calibration assumptions in automated decision governance pipelines. The system:

- Stores a calibration baseline snapshot for each operational domain at the time of system calibration, together with a unique snapshot identifier and a precise timestamp.
- At each subsequent evaluation cycle, computes a weighted drift score comparing current governance signal values against the stored baseline.
- Applies a strict three-priority blocking policy that evaluates, in order: (a) non-finite signal detection, (b) critical staleness of the calibration snapshot, and (c) weighted drift exceeding the configured threshold.
- Issues governance certifications that embed the calibration snapshot identifier, enabling retrospective traceability and invalidation of certifications issued under subsequently-invalidated baselines.
- Distinguishes explicitly between pass-through (no baseline available — not certified) and certified (baseline present and all checks passed) states, preventing downstream systems from treating absence of a baseline as implicit approval.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

I. SYSTEM ARCHITECTURE AND PIPELINE POSITION

The Assumption Validity Monitor (hereinafter "AVM" or "the System") occupies a specific architectural position within the automated decision governance pipeline. In the preferred embodiment, the AVM operates as the first evaluation stage after signal ingestion and before all subsequent governance checkpoints:

Signals → AVM → Context Admission Gate → Sequential Checkpoint Pipeline → Decision

This position is architecturally critical. By evaluating calibration validity before any checkpoint evaluation, the System ensures that no checkpoint can issue a governance certification under invalid calibration assumptions. A governance pipeline that evaluates signal quality, risk exposure, and coherence before verifying that calibration assumptions are still valid is structurally incomplete — it may produce technically valid certifications that are substantively unreliable.

Domain Isolation: In one embodiment, the System maintains independent calibration snapshots for each operational domain (e.g., one snapshot for cryptocurrency trading, a separate snapshot for credit risk evaluation, a separate snapshot for energy grid dispatch). Drift computation and blocking policy are applied independently per domain, ensuring that calibration drift in one domain does not affect governance in other domains.

The System is applicable to any domain in which automated decisions are made based on calibrated parameters, including but not limited to: financial trading, credit assessment, clinical decision support, autonomous robotics, energy grid management, insurance underwriting, and autonomous agent orchestration platforms.

II. CALIBRATION SNAPSHOT MODULE

II.A. Snapshot Structure

A calibration snapshot is a structured data record capturing the values of governance signals at the time of calibration for a specific operational domain. In one embodiment, the snapshot comprises:

- **snapshot_id:** A universally unique identifier (UUID) generated at snapshot creation time. This identifier is embedded in all governance certifications issued while the snapshot is active, enabling retrospective linkage between certifications and the calibration baseline under which they were issued.
- **domain:** A string identifier for the operational domain to which this snapshot applies.
- **timestamp:** A UTC ISO-8601 timestamp recording the precise moment of snapshot creation.
- **signals:** A dictionary mapping signal names to their baseline values. In one embodiment, the signals recorded include: probability_score, signal_coherence, risk_exposure, stress_resilience, trend_persistence, and logic_consistency. In other embodiments, any set of governance signals relevant to the deployment domain may be recorded.
- **version:** A human-readable version string for the calibration event.
- **metadata:** An extensible dictionary for deployment-specific annotations.

In other embodiments, the snapshot structure may include additional fields appropriate to the deployment domain, including regulatory metadata, model version identifiers, data source provenance, or environmental condition descriptors.

II.B. Snapshot Storage

In the preferred embodiment, snapshots are stored as JSON files in a designated snapshot directory, with one file per domain. The file naming convention encodes the domain identifier to enable efficient retrieval.

In other embodiments, snapshots may be stored in any persistent storage medium, including relational databases, document stores, distributed ledger systems, or encrypted vaults, provided that the storage mechanism supports: (a) atomic read/write operations, (b) preservation of the timestamp and snapshot identifier, and (c) retrieval of the most recent snapshot for a given domain.

Fallback to in-memory storage: In one embodiment, if filesystem storage is unavailable (e.g., read-only container environment), the System falls back to in-memory snapshot storage. The in-memory fallback preserves all snapshot functionality for the duration of the current process instance but does not persist across process restarts.

II.C. Parameter Versioning and Certification Linkage

Each calibration snapshot receives a unique snapshot_id at creation time. This identifier is embedded in governance certifications issued while the snapshot is active. If a calibration snapshot is subsequently determined to be invalid (e.g., due to discovery of data corruption, model errors, or retrospective analysis of market conditions), all governance certifications that embed the invalidated snapshot_id can be identified and flagged for review.

This capability directly addresses the regulatory requirement for governance audit trails that are traceable to specific calibration baselines.

III. DRIFT COMPUTATION MODULE

III.A. Weighted Drift Score

The AVM computes a weighted drift score comparing current governance signal values against the stored calibration baseline. The weighted drift score is defined as:

weighted_drift = Σ [weight(signal_i) × |current(signal_i) − baseline(signal_i)|]

where the summation is over all signals present in the calibration snapshot, weight(signal_i) is the configured weight for signal i, current(signal_i) is the current observed value of signal i, and baseline(signal_i) is the value of signal i recorded in the calibration snapshot.

In one embodiment, the signal weights are configured as follows:

- probability_score: 0.25
- signal_coherence: 0.25
- risk_exposure: 0.20
- stress_resilience: 0.15
- trend_persistence: 0.10
- logic_consistency: 0.05

These weights reflect the relative governance importance of each signal in the preferred deployment context. In other embodiments, the weights may be configured to any values appropriate to the deployment domain and signal importance hierarchy, provided that the weights are non-negative and sum to 1.0.

III.B. Threshold Tightening Under Stale Conditions

In one embodiment, the System automatically tightens the drift threshold when the calibration snapshot age exceeds the maximum age threshold but has not yet reached the critical age threshold. This tightening mechanism reflects the increased uncertainty associated with older calibration baselines.

In one embodiment, the tightened threshold is computed as:

effective_threshold = drift_threshold × (1 − tightening_factor)

where the tightening_factor is in one embodiment set to 0.25, reducing the effective threshold by 25% when the snapshot is stale but not yet critical. In other embodiments, the tightening factor may be set to any value in (0, 1) appropriate to the rate of change expected in the deployment domain.

III.C. Per-Signal Drift Components

In one embodiment, the System records the individual drift contribution of each signal alongside the aggregate weighted drift score. This per-signal decomposition enables:

- Identification of which specific signals are driving calibration drift
- Targeted recalibration of individual signal parameters without full system recalibration
- Audit documentation showing the specific evidence base for an AVM blocking decision

IV. THREE-PRIORITY BLOCKING POLICY

The AVM applies blocking checks in strict priority order. The first check that triggers a block produces an immediate BLOCK result — subsequent checks are not evaluated. This strict priority ordering is architecturally critical: it prevents a system with non-finite inputs from being evaluated for drift, and prevents a critically stale system from being evaluated for drift magnitude.

IV.A. Priority 1 — Non-Finite Signal Detection (ADR-075)

The System checks all input signals for non-finite values (NaN, positive infinity, negative infinity) before any other evaluation. If any input signal contains a non-finite value, the System issues an immediate BLOCK result with reason NON_FINITE_SIGNAL.

Critical safety rationale: In Python and most programming environments, the comparison `NaN > threshold` evaluates to `False`. A governance system that does not explicitly check for NaN inputs will silently pass a proposed action when signal inputs are corrupted or missing — treating the absence of a valid signal value as evidence of compliance. The present invention explicitly detects non-finite inputs and treats them as a blocking condition rather than allowing silent pass-through.

This behavior is fail-closed: any signal corruption produces a BLOCK, never a PASS. In other embodiments, the non-finite detection may be applied to any numerical input type (float, double, decimal) with platform-appropriate non-finite value checks.

IV.B. Priority 2 — Critical Staleness Detection

The System checks the age of the active calibration snapshot against a configurable critical age threshold. In one embodiment, the critical age threshold is set to 720 hours (30 days). If the snapshot age exceeds the critical age threshold, the System issues an immediate BLOCK result with reason CRITICAL_STALE, regardless of the current drift score.

Rationale: A governance system calibrated more than the critical age threshold ago cannot be considered a valid basis for certification of current decisions. The conditions under which it was calibrated may have changed fundamentally. No drift computation is meaningful if the baseline itself is too old to be a valid reference point.

In other embodiments, the critical age threshold may be set to any value appropriate to the rate of change expected in the deployment domain. For example, in highly volatile financial markets, a shorter critical age threshold may be appropriate; in stable regulatory compliance domains, a longer threshold may be appropriate.

IV.C. Priority 3 — Weighted Drift Threshold Evaluation

If neither the non-finite check nor the critical staleness check triggers a block, the System evaluates the weighted drift score against the effective threshold (which may be tightened if the snapshot is stale but not critical). If the weighted drift score exceeds the effective threshold, the System issues a BLOCK result with reason DRIFT_BLOCK.

In one embodiment, the default drift threshold is 35.0 (on a 0–100 scale), meaning that a combined weighted deviation across all signals of more than 35 points from the calibration baseline triggers a BLOCK. In other embodiments, this threshold may be set to any value appropriate to the deployment domain and acceptable calibration tolerance.

IV.D. Pass State and Certification

If all three checks pass without triggering a block, the System issues a PASS result, indicating that calibration assumptions remain sufficiently valid to support governance certification. The PASS result embeds the active snapshot_id, enabling downstream systems to link the certification to its calibration baseline.

V. PASS-THROUGH SEMANTICS AND EPISTEMIC TRANSPARENCY

A critical design principle of the System is the explicit distinction between two non-blocking states:

CERTIFIED (pass_through=False, is_valid=True): The System evaluated the current conditions against a baseline snapshot and determined that calibration assumptions remain valid. Downstream systems may treat this state as positive certification.

PASS-THROUGH (pass_through=True): The System had no baseline snapshot to compare against — either because no snapshot has been recorded for this domain, or because the AVM is disabled. This state does NOT indicate that the System has certified the decision. Downstream systems MUST treat pass_through=True as NON-CERTIFIED and apply appropriate conservatism.

Epistemic Transparency Principle: The absence of evidence (no baseline available) is not equivalent to evidence of compliance. A governance system that treats "no baseline" as "passed" commits a logical error that may result in dangerous false certifications. The present invention explicitly distinguishes these states and mandates conservative treatment of pass-through conditions by downstream systems.

VI. ADVERSARIAL SCENARIOS AND FAIL-CLOSED POLICY

The System is designed to maintain correct behavior under the following adversarial scenarios, each of which represents a real-world system failure mode:

Scenario 1 — Terra/LUNA Collapse Pattern: A governance system calibrated during stable market conditions (stablecoin peg maintained, high liquidity) continues to issue certifications as conditions deteriorate rapidly. The AVM detects accelerating drift across multiple signals (risk_exposure rising, signal_coherence falling, stress_resilience collapsing) and issues DRIFT_BLOCK before conditions reach catastrophic failure. In one embodiment, the test case models a 72-hour collapse scenario demonstrating that drift detection triggers blocking within the first 24 hours of accelerating divergence from baseline.

Scenario 2 — Gradual Drift Manipulation: An adversary attempts to evade drift detection by incrementally adjusting a single signal by small amounts over many cycles. The weighted drift score accumulates contributions from multiple small

deviations that individually fall below detection thresholds but collectively exceed the drift threshold. The System correctly detects and blocks on aggregate drift.

Scenario 3 — Boundary Manipulation: An adversary submits signal values precisely at the drift threshold boundary to probe the system's blocking behavior. The System applies deterministic threshold comparison without rounding or approximation, ensuring consistent blocking behavior at boundary conditions.

Scenario 4 — NaN Injection: An adversary or malfunctioning upstream system submits NaN or Inf values for one or more governance signals, attempting to cause silent pass-through due to Python's NaN comparison semantics. The System detects non-finite values at Priority 1 and issues immediate BLOCK, preventing silent pass-through.

Internal Exception Policy: Any internal exception within the AVM module propagates to the calling pipeline. The pipeline-level exception handler must treat any AVM exception as BLOCK (fail-closed). Exceptions do NOT produce pass-through results. This ensures that a malfunctioning AVM module does not silently allow decisions to proceed unchecked.

VII. ALTERNATIVE EMBODIMENTS

7.1 Adaptive Weight Adjustment: In one embodiment, signal weights are adjusted dynamically based on recent signal variance, increasing the weight of signals that have shown high variance in recent history and decreasing the weight of stable signals.

7.2 Multi-Baseline Comparison: In one embodiment, the System maintains multiple calibration baselines corresponding to different regime states (e.g., bull market baseline, bear market baseline, volatile baseline) and selects the most appropriate baseline for drift comparison based on the currently classified regime.

7.3 Continuous Snapshot Updating: In one embodiment, the System implements a rolling baseline update mechanism that periodically updates the calibration snapshot while maintaining the previous snapshot for comparison purposes, enabling detection of both instantaneous and cumulative drift.

7.4 Distributed Snapshot Consensus: In one embodiment deployed in multi-node architectures, calibration snapshots are replicated across nodes with a consensus mechanism ensuring that all nodes operate with identical baseline snapshots.

7.5 Domain-Specific Signal Sets: In non-financial embodiments, the set of governance signals tracked by the AVM may differ. For example, in a clinical decision support deployment, signals might include diagnostic confidence score, data completeness score, patient population match score, and guideline currency score. The drift computation and blocking policy architecture is identical regardless of the specific signals tracked.

CLAIMS

Claim 1 (Broad — Core AVM) — A computer-implemented system for monitoring calibration validity in an automated decision pipeline, comprising: a snapshot storage module that records a calibration baseline comprising a plurality of governance signal values at a first point in time; a drift computation module that computes a weighted drift score by comparing current governance signal values against said calibration baseline; a blocking module that blocks execution of a proposed action when said weighted drift score exceeds a configured threshold; and a certification module that embeds a calibration snapshot identifier in governance certifications issued when the drift score is below the threshold.

Claim 2 (Broad — Three-Priority Policy) — The system of Claim 1, wherein the blocking module applies blocking checks in strict priority order: a first check detecting non-finite values in any governance signal input; a second check detecting that the calibration baseline has exceeded a critical age threshold; and a third check evaluating the weighted drift score against the effective threshold; wherein the first triggered check produces an immediate block result and subsequent checks are not evaluated.

Claim 3 (Specific — NaN Fail-Closed) — The system of Claim 2, wherein the first priority check detects NaN, positive infinity, and negative infinity values in governance signal inputs and issues an immediate block result upon detection of any non-finite value, preventing silent pass-through caused by NaN comparison semantics in which NaN compared against any numerical threshold evaluates to False.

Claim 4 (Specific — Critical Age Block) — The system of Claim 2, wherein the second priority check compares the age of the calibration baseline against a configurable critical age threshold and issues an unconditional block result when said age is exceeded, regardless of the computed drift score.

Claim 5 (Specific — Stale Threshold Tightening) — The system of Claim 1, wherein the drift computation module applies a tightened effective threshold when the calibration baseline age exceeds a maximum age threshold but has not yet reached the critical age threshold, said tightened threshold being computed as a fraction of the configured drift threshold, wherein older baselines require lower drift scores to trigger blocking.

Claim 6 (Broad — Pass-Through Semantics) — The system of Claim 1, further comprising an epistemic transparency module that explicitly distinguishes between a certified state in which the system evaluated current conditions against a valid baseline and determined compliance, and a pass-through state in which no valid baseline was available for comparison; wherein downstream systems are required to treat the pass-through state as non-certified rather than as implicitly approved.

Claim 7 (Specific — Snapshot Versioning) — The system of Claim 1, wherein each calibration baseline snapshot is assigned a unique identifier embedded in governance certifications issued while said snapshot is active, enabling retrospective identification and invalidation of certifications issued under a subsequently-invalidated calibration baseline.

Claim 8 (Broad — Domain Isolation) — The system of Claim 1, wherein independent calibration baseline snapshots are maintained for each of a plurality of operational domains, and wherein drift computation and blocking policy are applied independently per domain such that calibration drift in one domain does not affect governance certification in other domains.

Claim 9 (Broad — System Claim) — A computer-implemented system configured to enforce a fail-closed blocking policy in an automated decision governance pipeline by: storing a point-in-time calibration snapshot of governance signal values; detecting non-finite signal inputs and blocking execution before any threshold comparison; detecting critical staleness of the calibration snapshot and blocking execution regardless of current signal values; computing a weighted multi-signal drift score; and blocking execution when said drift score exceeds a domain-configurable threshold, wherein the system treats all failure modes as blocking conditions rather than pass-through conditions.

Claim 10 (Broad — Method Claim) — A computer-implemented method for fail-closed calibration drift detection in an automated decision system, comprising: recording a calibration baseline snapshot of governance signal values at a first time; receiving current governance signal values at a subsequent time; detecting non-finite values in said current signal values and blocking execution upon detection; detecting critical age of said calibration baseline and blocking execution upon detection; computing a weighted sum of absolute deviations between current signal values and baseline signal values; and blocking execution when said weighted sum exceeds a configured threshold.

ABSTRACT

An automated calibration drift detection and fail-closed blocking system monitors the continued validity of calibration assumptions in automated decision governance pipelines. The system stores a calibration baseline snapshot of governance signal values at calibration time, assigns a unique snapshot identifier for certification provenance tracking, and applies a three-priority blocking policy at each subsequent evaluation: first detecting non-finite signal inputs that would cause silent pass-through due to NaN comparison semantics; second detecting critical staleness of the calibration baseline; and third computing a weighted multi-signal drift score and blocking when it exceeds a configurable threshold. The system explicitly distinguishes certified states from pass-through states, preventing downstream systems from treating absence of a baseline as implicit approval. Independent calibration snapshots are maintained per operational domain. The system is applicable to financial trading, clinical decision support, autonomous robotics, energy grid management, and any automated decision domain in which governance certifications must be traceable to valid calibration baselines. To the inventor's knowledge, this is the first system to enforce a fail-closed three-priority blocking policy combining non-finite signal detection, critical age detection, and weighted multi-signal drift scoring as independent blocking conditions with explicit certification provenance tracking.

DRAWINGS

FIG. 1 — AVM Architecture Pipeline Position

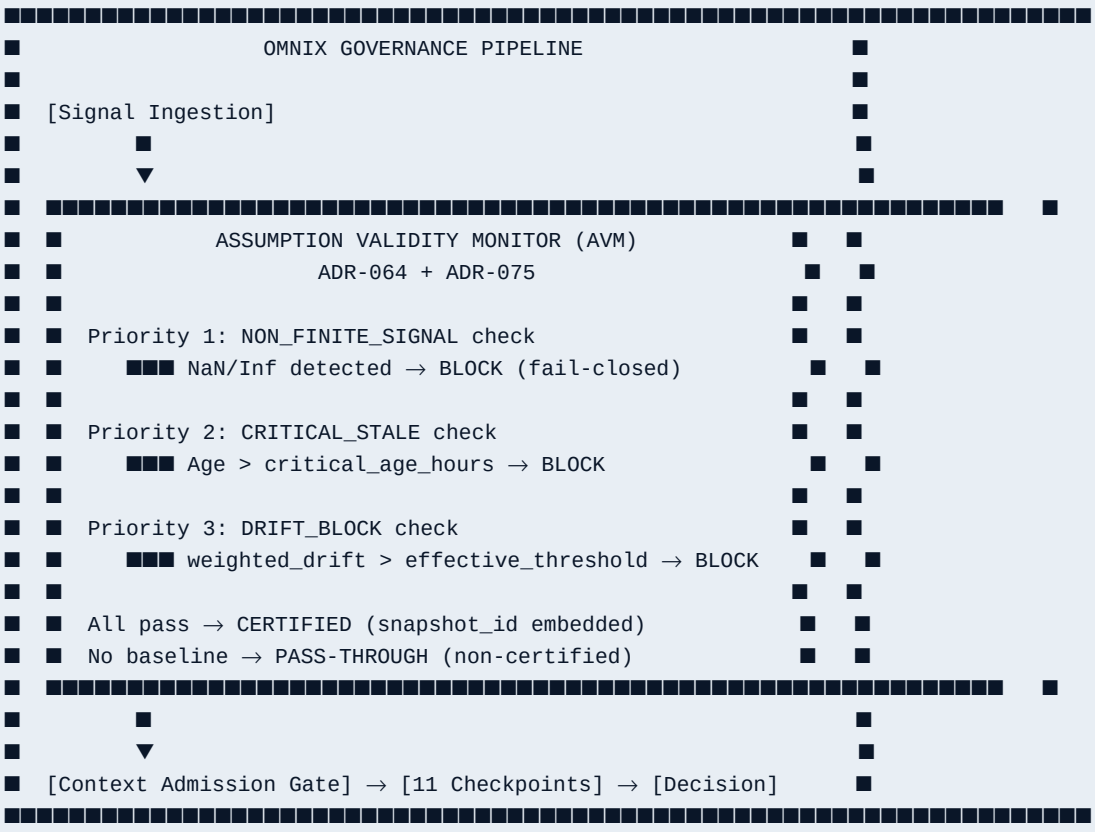
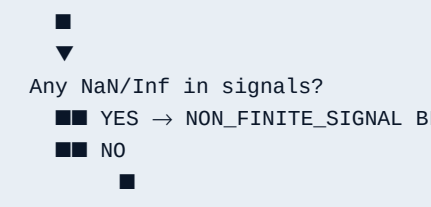


FIG. 2 — Weighted Drift Computation

Signal	Weight	|current − baseline|	Contribution
probability_score	0.25	× Δ_P	= 0.25 × Δ_P
signal_coherence	0.25	× Δ_C	= 0.25 × Δ_C
risk_exposure	0.20	× Δ_R	= 0.20 × Δ_R
stress_resilience	0.15	× Δ_S	= 0.15 × Δ_S
trend_persistence	0.10	× Δ_T	= 0.10 × Δ_T
logic_consistency	0.05	× Δ_L	= 0.05 × Δ_L

weighted_drift = sum of all contributions (0–100 scale)
In one embodiment: threshold = 35.0 (configurable)
Stale tightening: effective_threshold = 35.0 × (1 − 0.25) = 26.25

FIG. 3 — Blocking Policy Decision Tree




▼
Snapshot age > critical_age_hours?
■■ YES → CRITICAL_STALE BLOCK
■■ NO
■
▼
No snapshot exists?
■■ YES → PASS-THROUGH (non-certified)
■■ NO
■
▼
weighted_drift > effective_threshold?
■■ YES → DRIFT_BLOCK
■■ NO → CERTIFIED (snapshot_id embedded in receipt)